

1. Что такое Spring AI, его возможности и какую проблему он решает

Spring AI – это расширение фреймворка Spring. Он предоставляет набор абстракций и инструментов, которые упрощают интеграцию больших языковых моделей (LLM) и других AI-сервисов в Java-приложения.

Spring AI решает несколько ключевых проблем:

1 Отсутствие единого стандарта. Spring AI предоставляет единые абстракции, поэтому можно сменить модель или провайдера, изменив только конфигурацию, а не весь код приложения.

2 Сложность работы с LLM в Java. Так как Большинство AI-инструментов ориентированы на Python. А Spring AI делает работу с ии привычной для разработчиков Spring. Например, упрощает обработку данных, сопоставляя выходные данные моделей ИИ с объектами Java (POJO).

3 Автоконфигурация Spring Boot. Spring AI поддерживает автоконфигурацию для ряда популярных моделей. Полезно для моделей, где требуются много модельные конфигурации

4 Поддерживаемые модели: Anthropic, Azure OpenAI, Amazon Bedrock, Google, HuggingFace, Mistral, Oracle, Stability AI, Watson, Minimax, Moonshot, QianFan, ZhiPu AI, PostgresML, and ONNX Transformers.

Возможности Spring AI

1 AI-чат и ответы на вопросы. Создание интеллектуальных чат-систем и поиск информации с помощью LLM.

2 Семантический поиск (RAG). Поиск данных в документах с использованием векторных баз данных.

3 Генерация изображений. Spring AI поддерживает работу с генеративными моделями, которые создают изображения на основе текстовых запросов.

4 Работа с аудио. Также позволяет использовать аудиомодели для преобразования текста в речь, создания синтетических голосов или генерации звуков.

5 Мониторинг работы ИИ. Встроенные инструменты для отслеживания и анализа работы ИИ, включая метрики, логи и трассировки, что помогает контролировать производительность приложения.

6 Поддерживаемые векторные БД: Apache Cassandra, Azure Vector Search, Chroma, Milvus, MongoDB Atlas, Neo4j, Oracle, PostgreSQL/PGVector, PineCone, Qdrant, Redis, and Weaviate.

2. Основные компоненты: ChatClient, Advisors, Chat Memory, VectorStore, Embedding Models

В Spring AI есть несколько ключевых компонентов и абстракций, которые позволяют удобно работать с AI-моделями в Java-приложениях. Такие как:

ChatClient – для реализации чата: управление последовательностью сообщений, ролями участников и потоковой генерацией ответов.

EmbeddingClient – для преобразования текста в векторные представления, используемые в семантическом поиске и сравнении документов.

MultiModalClient – для работы с несколькими ai моделями, объединяющими текст, изображения и другие типы медиа.

AudioClient – для задач распознавания речи и транскрипции аудио.

ImageClient – для генерации и обработки изображений.

VectorStore – для хранения и поиска эмбеддингов, обеспечивающий возможность семантического поиска.

PromptTemplate – для создания динамических промптов с использованием переменных.

TokenCounter – для оценки количества токенов с целью контроля затрат и соблюдения ограничений контекста модели.

ChatClient

ChatClient – это основной интерфейс, через который приложение взаимодействует с языковой моделью. Он обычно создаётся с помощью **ChatClient.Builder**. В большинстве случаев Spring Boot автоматически создаёт и настраивает этот объект, поэтому разработчику достаточно внедрить его в класс и использовать для работы с моделью. Если приложение использует несколько моделей, можно создать несколько отдельных **ChatClient**, каждый для своей модели.

Работа **ChatClient** основана на сообщениях двух типов. Первый тип — это **user messages**, то есть сообщения пользователя, которые содержат основной запрос. Второй тип – **system messages**. Они формируются системой и задают правила или контекст работы модели. Затем **.user()** добавляет сообщение пользователя. После этого вызывается **.call()**, который отправляет запрос к AI-модели, а **.content()** возвращает текстовый ответ.

После формирования запроса **ChatClient** позволяет получить ответ несколькими способами. Самый простой вариант – получить текстовый ответ. Однако можно получить и более подробный объект ответа: **ChatResponse**, который содержит дополнительную информацию, например метаданные генерации и количество использованных токенов. Также можно вернуть структурированные данные, а приложение сразу получает их в виде POJO, без ручного парсинга текста. И 3 вариант позволяет вернуть асинхронный ответ, в виде потока.

Advisors

Это механизм, который позволяет перехватывать и изменять запросы и ответы от модели.

Они позволяют:

- добавлять контекст (например, данные или историю)
- изменять **prompt** перед отправкой в модель
- постобрабатывать результат

Ключевая особенность — цепочка выполнения. Порядок **advisors** влияет на итоговый результат.

Chat Memory

Chat Memory — это механизм хранения истории диалога между запросами. В Spring AI для этого используется интерфейс ChatMemory, который определяет базовые операции работы с памятью: добавление сообщений, получение истории и её очистку. Необходимость такого механизма связана с тем, что языковые модели не сохраняют состояние между вызовами, поэтому весь контекст взаимодействия должен передаваться при каждом запросе.

В текущей версии Spring AI доступна встроенная реализация MessageWindowChatMemory. Она представляет собой память с ограниченным размером окна, которое по умолчанию составляет 20 сообщений. При превышении этого лимита более старые сообщения удаляются, что позволяет контролировать использование памяти. При этом системные сообщения обрабатываются отдельно: они сохраняются, но при добавлении нового системного сообщения предыдущие системные сообщения удаляются. Такой подход обеспечивает наличие актуального контекста при каждом обращении к модели.

Интеграция памяти в Spring AI реализуется через Advisors, которые управляют тем, как именно история диалога добавляется в запрос. Основные варианты включают MessageChatMemoryAdvisor, который добавляет историю как список сообщений, PromptChatMemoryAdvisor, который вставляет её в системный prompt в виде текста, и VectorStoreChatMemoryAdvisor, использующий векторное хранилище. При вызове ChatClient память обрабатывается автоматически: история извлекается по идентификатору диалога (conversationId) и включается в текущий запрос без необходимости ручного управления.

На практике хранение памяти разделяется на два уровня: логика управления (ChatMemory) и физическое хранилище (ChatMemoryRepository). По умолчанию используется in-memory реализация, однако возможны и другие варианты, такие как JDBC или NoSQL-хранилища.

Для демонстрации возможностей Spring AI было разработано тестовое приложение. В нём используется ChatClient с подключёнными advisors: для памяти – MessageChatMemoryAdvisor, и для RAG – QuestionAnswerAdvisor. Также настроен VectorStore, в который загружается документ, и ChatMemory для хранения истории диалога. Это позволяет показать работу как обычного чата, так и чата с использованием RAG (про который поговорим дальше).

RAG

RAG (Retrieval-Augmented Generation) – это подход, при котором языковой модели добавляется контекст для формирования более точного ответа.

Процесс работы выглядит так:

1. Документ разбивается на части (чанки)
2. Каждый чанк преобразуется в вектор (embedding)
3. Векторы сохраняются в векторные базы данных
4. При запросе пользователя:

- запрос тоже превращается в вектор
- находится несколько самых похожих чанков
- они передаются в модель как контекст

5. Модель генерирует ответ с учетом найденной информации

Таким образом, модель отвечает не просто на основе обучения, а с опорой на конкретный документ.

VectorStore и Embedding Models

VectorStore и Embedding Models — это основа работы RAG в Spring AI.

Векторные базы данных хранят текст не в обычном виде, а в виде числовых представлений – эмбедингов. Эти векторы можно представить как точки в пространстве: чем ближе точки друг к другу, тем ближе их смысл.

Embedding Model – это интерфейс, который преобразует текст в такой вектор. Он является частью общей Model API Spring AI и использует стандартные классы EmbeddingRequest и EmbeddingResponse для работы с входными и выходными данными.

На практике разработчик не всегда напрямую работает с EmbeddingModel. В большинстве случаев он используется внутри VectorStore и настраивается автоматически через автоконфигурацию Spring Boot. Например, при использовании Ollama применяется OllamaEmbeddingModel.

Основные составляющие конечной структуры в векторной БД:

ID — уникальный идентификатор объекта.

Vector (Embedding) — сам вектор, который хранится в специальной структуре для быстрого поиска ближайших соседей.

Payload (Metadata) — дополнительные данные в формате ключ-значение.

Оригинальный текст — сам фрагмент или ссылка на внешний источник.

От чего зависит качество ответов в RAG

Качество ответов при использовании RAG определяется не только самой языковой моделью, но и тем, насколько правильно организован процесс извлечения данных из внешних источников. Даже при использовании мощной LLM итоговый ответ может быть неточным или неполным, если система поиска работает некорректно.

Одним из ключевых этапов является подготовка контекста, который будет передан модели. Важно не только наличие данных, но и то, в каком виде они представлены. Контекст должен быть структурирован таким образом, чтобы модель могла легко извлекать из него смысл и использовать его для генерации ответа.

Одним из ключевых факторов является разбиение документа на фрагменты (chunks). Если чанки слишком большие, в них может содержаться лишняя информация, и модель будет получать нерелевантный контекст. Если же чанки слишком маленькие, теряется смысловая связность текста, и поиск начинает возвращать фрагменты, не содержащие полного ответа. Поэтому важно подобрать оптимальный размер чанка и учитывать границы предложений или абзацев, чтобы сохранять целостность смысла.

Даже при правильно выбранном размере чанков система может возвращать нерелевантные фрагменты, если поиск работает неточно. Чем

лучше модель отражает семантику текста, тем более релевантные чанки будут извлекаться.

Было проведено сравнение ответов при разных параметрах разбиения документа. Модели был задан один вопрос: “Какие проблемы решает SpringAI”. Из проведенного сравнения получилось, что для больших чанков модель не выделяет явные проблемы, описанные в контексте, а вместо этого интерпретирует технические возможности системы как решаемые задачи. Это связано с тем, что при использовании больших чанков в контекст попадает избыточная информация, и модель теряет фокус на ключевом фрагменте текста. Для маленьких чанков такая же проблема, но здесь просто извлечен неправильный контекст.

Советы и антипаттерны при работе с Spring AI

При работе со Spring AI важно понимать, что сам фреймворк решает задачу интеграции с моделями, но не гарантирует качество итогового поведения системы, оно определяется архитектурой приложения и подготовкой данных.

Во-первых, важно правильно управлять контекстом. Языковые модели не имеют памяти в привычном смысле и работают только с теми данными, которые переданы в текущем запросе. Это означает, что каждый вызов модели – это фактически новая сборка контекста: system prompt, сообщение пользователя, история диалога, RAG-фрагменты и результаты инструментов. Если этот контекст перегружен или плохо структурирован, модель начинает терять фокус. Также следует внимательно относиться к использованию Advisors. Порядок их подключения напрямую влияет на результат. Например, сначала должна применяться память (Chat Memory), а затем поиск по Vector Store. Если изменить порядок, модель может работать некорректно: либо не учитывать историю диалога, либо игнорировать найденный контекст.

Во-вторых, необходимо учитывать ограничения контекстного окна. В реальных сценариях в prompt одновременно попадают история, RAG-документы и дополнительные данные, что может привести к перегрузке контекста и ухудшению качества ответов. Поэтому важно ограничивать объём передаваемой информации и использовать подходы вроде сокращения истории или отбора наиболее релевантных данных.

При использовании RAG одной из ключевых рекомендаций является рассматривать его не как отдельную функцию, а как пайплайн обработки данных. В него входят этапы индексации, разбиения документов, генерации эмбеддингов, поиска, фильтрации и постобработки результатов. Ошибки на любом из этих этапов приводят к ухудшению качества ответа.

Частым антипаттерном является ожидание, что Vector Store всегда будет возвращать точные результаты. На практике поиск может возвращать “похожие, но не релевантные” фрагменты, особенно если документы плохо подготовлены или параметры поиска не настроены. Поэтому важно проверять, какие именно данные попадают в контекст модели.

Также стоит избегать использования “сырых” документов. Как показывает практика, качество RAG сильно зависит от предварительной

обработки данных: структуры текста, удаления шума и логического разбиения на части. Необработанные документы приводят к ухудшению поиска и увеличению количества нерелевантных ответов.

Таким образом, при работе со Spring AI основное внимание следует уделять не только настройке компонентов, но и подготовке данных, управлению контекстом и проверке реального поведения системы.